```
import torch
import torch.nn as nn
```

```
class Model(nn.Module):
  def __init__(self, num_features):
    super().__init__()
    self.linear = nn.Linear(num_features, 1)
    self.sigmoid = nn.Sigmoid()

  def forward(self, features):
    out = self.linear(features)
    out = self.sigmoid(out)
    return out
```

```
# create dataset
features = torch.rand(10, 5)

# create model
model = Model(5)

# call model
model(features)
```

```
tensor([[0.3453],
        [0.3872],
        [0.3541],
        [0.4077],
        [0.3864],
        [0.3240],
        [0.4125],
        [0.3546],
        [0.3372],
        [0.3435]], grad_fn=<SigmoidBackward0>)
```

```
model.linear.weight
```

```
Parameter containing:
tensor([[-0.1968,  0.2603, -0.1308, -0.3409,  0.0152]], requires_grad=True)
```

```
model.linear.bias
```

```
Parameter containing:
tensor([-0.3163], requires_grad=True)
```

```
!pip install torchinfo
```

```
Collecting torchinfo
  Downloading torchinfo-1.8.0-py3-none-any.whl.metadata (21 kB)
Downloading torchinfo-1.8.0-py3-none-any.whl (23 kB)
Installing collected packages: torchinfo
Successfully installed torchinfo-1.8.0
```

```
from torchinfo import summary
```

```
summary(model, input_size = (10, 5))
```

```
==========================================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================================
Model                                    [10, 1]                   --
├─Linear: 1-1                            [10, 1]                   6
├─Sigmoid: 1-2                           [10, 1]                   --
==========================================================================================
Total params: 6
Trainable params: 6
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 0.00
==========================================================================================
Input size (MB): 0.00
Forward/backward pass size (MB): 0.00
Params size (MB): 0.00
Estimated Total Size (MB): 0.00
==========================================================================================
```

Neural network with hidden layer

```
import torch
import torch.nn as nn

class Model(nn.Module):
  def __init__(self, num_features):
    super().__init__()
    self.linear1 = nn.Linear(num_features, 3)
    self.relu = nn.ReLU()
    self.linear2 = nn.Linear(3, 1)
    self.sigmoid = nn.Sigmoid()

  def forward(self, features):
    out = self.linear1(features)
    out = self.relu(out)
    out = self.linear2(out)
    out = self.sigmoid(out)
    return out
```

```
features = torch.rand(10, 5)
model = Model(features.shape[1])

model(features)
```

```
tensor([[0.5652],
        [0.5652],
        [0.5652],
        [0.5652],
        [0.5652],
        [0.5668],
        [0.5652],
        [0.5671],
        [0.5652],
        [0.5652]], grad_fn=<SigmoidBackward0>)
```

```
summary(model, input_size = (10, 5))
```

```
==========================================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================================
Model                                    [10, 1]                   --
├─Linear: 1-1                            [10, 3]                   18
├─ReLU: 1-2                              [10, 3]                   --
├─Linear: 1-3                            [10, 1]                   4
├─Sigmoid: 1-4                           [10, 1]                   --
==========================================================================================
Total params: 22
Trainable params: 22
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 0.00
==========================================================================================
Input size (MB): 0.00
Forward/backward pass size (MB): 0.00
Params size (MB): 0.00
Estimated Total Size (MB): 0.00
==========================================================================================
```

Using sequential to make it more smaller and efficient

```
import torch
import torch.nn as nn

class Model(nn.Module):
  def __init__(self, num_features):
    super().__init__()

    self.network = nn.Sequential(
        nn.Linear(num_features, 3),
        nn.ReLU(),
        nn.Linear(3, 1),
        nn.Sigmoid()
    )

  def forward(self, features):
    out = self.network(features)
    return out
```

```
features = torch.rand(10, 5)
model = Model(features.shape[1])

model(features)
```

```
tensor([[0.5303],
        [0.5318],
        [0.5333],
        [0.5204],
        [0.5268],
        [0.5179],
        [0.4955],
        [0.5330],
        [0.5237],
        [0.5174]], grad_fn=<SigmoidBackward0>)
```

Improving our previous code with nn module

```
import numpy as np
import pandas as pd
import torch
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder
```

```
df = pd.read_csv('https://raw.githubusercontent.com/gscdit/Breast-Cancer-Detection/refs/heads/master/data.csv')
df.head()
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_m
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1

5 rows × 33 columns

```
df.shape
```

```
(569, 33)
```

```
df.drop(columns=['id', 'Unnamed: 32'], inplace= True)
```

```
X_train, X_test, y_train, y_test = train_test_split(df.iloc[:, 1:], df.iloc[:, 0], test_size=0.2)
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
encoder = LabelEncoder()
y_train = encoder.fit_transform(y_train)
y_test = encoder.transform(y_test)
```

```
X_train_tensor = torch.from_numpy(X_train.astype(np.float32))
X_test_tensor = torch.from_numpy(X_test.astype(np.float32))
y_train_tensor = torch.from_numpy(y_train.astype(np.float32))
y_test_tensor = torch.from_numpy(y_test.astype(np.float32))
```

```
X_train_tensor.shape
```

```
torch.Size([455, 30])
```

```
y_train_tensor.shape
```

```
torch.Size([455])
```

```
import torch.nn as nn


class MySimpleNN(nn.Module):

  def __init__(self, num_features):
    super().__init__()
    self.linear = nn.Linear(num_features, 1)
```

```
    self.sigmoid = nn.Sigmoid()

  def forward(self, features):
    out = self.linear(features)
    out = self.sigmoid(out)

    return out



  # def loss_function(self, y_pred, y):
  #   # Clamp predictions to avoid log(0)
  #   epsilon = 1e-7
  #   y_pred = torch.clamp(y_pred, epsilon, 1 - epsilon)

  #   # Calculate loss
  #   loss = -(y_train_tensor * torch.log(y_pred) + (1 - y_train_tensor) * torch.log(1 - y_pred)).mean()
  #   return loss
```

```
learning_rate = 0.1
epochs = 25
```

```
loss_function = nn.BCELoss()
```

```
# create model
model = MySimpleNN(X_train_tensor.shape[1])

# define optimizer
optimizer = torch.optim.SGD(model.parameters(), lr = learning_rate)

# define loop
for epoch in range(epochs):

  # forward pass
  y_pred = model(X_train_tensor)

  # loss calculate
  loss = loss_function(y_pred, y_train_tensor.reshape(-1, 1)) # we can use .view(-1, 1) as well

  optimizer.zero_grad()
  # backward pass
  loss.backward()
  optimizer.step()

  # print loss in each epoch
  print(f'Epoch: {epoch + 1}, Loss: {loss.item()}')
```

```
Epoch: 1, Loss: 0.7201232314109802
Epoch: 2, Loss: 0.549796998500824
Epoch: 3, Loss: 0.45970967411994934
Epoch: 4, Loss: 0.40346550941467285
Epoch: 5, Loss: 0.3643571734428406
Epoch: 6, Loss: 0.3352360725402832
Epoch: 7, Loss: 0.3125053942203522
Epoch: 8, Loss: 0.29414257407188416
Epoch: 9, Loss: 0.2789158523082733
Epoch: 10, Loss: 0.2660287022590637
Epoch: 11, Loss: 0.2549409568309784
Epoch: 12, Loss: 0.24527212977409363
Epoch: 13, Loss: 0.23674556612968445
Epoch: 14, Loss: 0.2291547656059265
Epoch: 15, Loss: 0.22234201431274414
Epoch: 16, Loss: 0.2161845713853836
Epoch: 17, Loss: 0.21058513224124908
Epoch: 18, Loss: 0.20546558499336243
Epoch: 19, Loss: 0.20076225697994232
Epoch: 20, Loss: 0.1964227259159088
Epoch: 21, Loss: 0.19240325689315796
Epoch: 22, Loss: 0.1886672079563141
Epoch: 23, Loss: 0.18518352508544922
Epoch: 24, Loss: 0.18192560970783234
Epoch: 25, Loss: 0.17887066304683685
```

```
# model evaluation
with torch.no_grad():
  y_pred = model.forward(X_test_tensor)
```

```
y_pred = (y_pred > 0.9).float()
accuracy = (y_pred == y_test_tensor).float().mean()
print(f'Accuracy: {accuracy.item()}')
```

```
Accuracy: 0.5572484135627747
```

Start coding or generate with AI.